

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Направление: «Фундаментальная информатика и информационные технологии»
ООП: «Программирование и информационные технологии»

ОТЧЁТ О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

Тема задания: Вероятностный апостериорный анализ трудоёмкости
MSD-сортировки.

Выполнил: Ерхов Александр Александрович, группа 23.Б12–ПУ

Руководитель НИР: доцент, кафедра моделирования электромеханических и
компьютерных систем, Никифоров К. А.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Постановка задачи	4
1 Априорный анализ	5
2 Алгоритм	6
2.1 Генератор входных данных	6
2.2 Реализация сортировки	6
2.3 Вычислительный эксперимент	8
3 Как пользоваться шаблоном	9
3.1 Вставка формул	9
3.2 Вставка каритнок	9
3.3 Вставка таблиц	10
3.4 Вставка литературы	10
4 Текст	10
4.1 Текст подраздела	11
ЗАКЛЮЧЕНИЕ	12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13

ВВЕДЕНИЕ

Поразрядная сортировка не нуждается в обсуждении ее актуальности и применимости. Её история, как и должно, глубокая и обширная. Ещё в конце XIX века Herman Hollerith создал механический табулятор, сортировщик, который использовался для переписи населения [1]

Обсуждения стоит именно вопрос исследования трудоёмкости алгоритма. LSD-сортировка понятна и предсказуема, а вот MSD-сортировка – классический пример алгоритм класса NPR, как и, например, быстрая сортировка. И обычно вопрос её трудоёмкости заканчивается худшим случаем $O(nk)$, где n – количество элементов, k – количество разрядов элемента, и лучшим случаем $\Omega(n \log_b n)$, b – верхнее значение одного разряда. (На каждом шаге по b корзин). Вопрос построения вероятностной модели, с последующим получением распределения с целью анализа среднего случая, является нетривиальным. Но в 2018 году ряд исследователей доказал центральную предельную теорему [2]

Цель – провести вероятностный апостериорный анализ трудоёмкости MSD-сортировки на основе результатов, полученных в [2]. Задачи:

1. Исследование материала [2]
2. Разработка алгоритма на основе модели, приведенной в [2]
3. Проведение вычислительного эксперимента
4. Проверка на соответствие выборочного среднего и математического ожидания
5. Проверка центральной предельной теоремы с помощью критерия Пирсона

Постановка задачи

Будем рассматривать словарь (массив слов). Пусть алфавит $\Sigma = \{0, 1\}$. Количество символов в слове непостоянно. Количество слов в словаре также заранее неизвестно – n . Необходимо отсортировать словарь в лексикографическом порядке.

Генерация словаря. Каждое слово генерируется независимо друг от друга. Начальный символ, в каждом слове, выбирается согласно начальному распределению μ , т. е. $P(\xi = 0) = \mu_0$, $P(\xi = 1) = \mu_1$, где $\mu_0 + \mu_1 = 1$. Далее используем марковские цепи: каждый следующий символ зависит только от предыдущего, в соответствии с матрицей переходов $P = [p_{ij}]$, $i, j \in \{0, 1\}$. При этом $p_{ij} \neq \frac{1}{2}, \forall i, j$ (Этот случай исключается как тривиальный). Также здесь введем понятие *энтропии*:

$$H = \sum_i \pi_i H_i, \text{ где } H_i = -\sum_j p_{ij} \log p_{ij}, \pi_0 = \frac{p_{10}}{p_{01} + p_{10}}, \pi_1 = \frac{p_{01}}{p_{01} + p_{10}}$$

Причины. Алгоритм поразрядной MSD-сортировки находит применение в задачах, где определяющим фактором является значение старших разрядов ключей. В отличие от LSD-сортировки, обрабатывающей данные от младших разрядов к старшим и предполагающей фиксированную длину ключей, метод MSD обеспечивает раннее разделение элементов по наиболее значимым позициям, что особенно эффективно при работе с переменной длиной данных, таких как строки или идентификаторы. Он используется в системах, где требуется лексикографическое упорядочивание. Применение алгоритма оправдано в случаях, когда важна иерархическая структура ключей и наблюдается неоднородность их длины или распределения.

1 Априорный анализ

Как я уже писал выше, этот анализ проведен не мной, в [2]. Здесь приведу результаты, без доказательств.

B_n^μ – это случайная величина, представляющая количество операций с корзинами (*bucket operations*) в алгоритме сортировки (или внешнюю длину пути в соответствующем *trie*¹) при сортировке n строк, сгенерированных марковским источником с начальным распределением μ . При апостериорном анализе это будет означать количество операций сравнения.

Итого:

$$\mathbb{E}[B_n^\mu] = \frac{1}{H} n \log n + O(n);$$

$$Var(B_n^\mu) = \sigma^2 n \log n + O(n\sqrt{\log n}), \text{ где}$$

$$\sigma^2 = \frac{\pi_0 p_{00} p_{01}}{H^3} \left(\log(p_{00}/p_{11}) + \frac{H_1 - H_0}{p_{01} + p_{10}} \right)^2 + \frac{\pi_1 p_{10} p_{11}}{H^3} \left(\log(p_{10}/p_{11}) + \frac{H_1 - H_0}{p_{01} + p_{10}} \right)^2;$$

$$\frac{B_n^\mu - \mathbb{E}[B_n^\mu]}{\sqrt{Var(B_n^\mu)}} \xrightarrow{d} \mathcal{N}(0, 1)$$

¹ Структура данных. Встречается как: Префиксное дерево, trie, prefix trie, ... [3]

2 Алгоритм

2.1 Генератор входных данных

Особенности были описаны в разделе "Постановка задачи". Символы будут разыгрываться по методу Монте-Карло. Код:

```
1 import random
2
3 def generator(
4     n: int,
5     k: int,
6     delta: int,
7     mu: float,
8     P: list[float],
9 ):
10     array = []
11     for _ in range(n):
12         if random.random() < mu:
13             word = '0'
14         else:
15             word = '1'
16         real_num_k = k + random.randint(-delta, delta)
17         for __ in range(real_num_k - 1):
18             if word[-1] == '0':
19                 if random.random() < P[0]:
20                     word = word + '0'
21                 else:
22                     word = word + '1'
23             else:
24                 if random.random() < P[1]:
25                     word = word + '0'
26                 else:
27                     word = word + '1'
28         array.append(word)
29     return array
```

Пояснения к коду. μ — *float*, потому что, на самом деле второе число не нужно. Это важно при теоретическом анализе, но для применения Монте-Карло в данном контексте достаточно одного числа, так как алфавит с размерностью 2. Аналогично с матрицей P , которая выродилась в вектор. В итоге получаем массив с n словами, каждое из которых длиной $k \pm \Delta$, в соответствии с требованиями описанными в разделе *Постановка задачи*

2.2 Реализация сортировки

Я реализовал *in-place* вариант MSD-сортировки, как классический:

```

1 def MSD_sort(
2     arr: list[str],
3     n: int
4 ) -> int:
5     stack = [(0, 0, n)] # digit, left, right
6     total_count = 0
7
8     while stack:
9         digit, left, right = stack.pop()
10        if right-left <= 1 or digit >= max(len(arr[i]) for i in range(left,
11        right)):
12            continue
13        count, groups = counting_sort(arr, left, right, digit)
14        total_count += count
15        for i in range(1, len(groups)):
16            new_left, new_right = groups[i]
17            if new_right - new_left > 1:
18                stack.append((digit+1, new_left, new_right))
19
20    return total_count

```

В процессе необходимо использовать устойчивую сортировку для сортировки отдельных разрядов. Я использовал *counting sort*, так как она приемлема для сортировки символов и оптимальна из-за скудности размера теоретически возможного алфавита:

```

1 def counting_sort(
2     arr: list[str],
3     left: int,
4     right: int,
5     target_digit: int
6 ) -> tuple[int, list[tuple[int, int]]]:
7     n = right - left
8     R = 3 # None 0 1
9     C = [0] * (R + 1)
10    res = 0 # count of operations
11
12    for i in range(left, right):
13        if target_digit >= len(arr[i]):
14            ch = 0
15        elif arr[i][target_digit] == '0':
16            res += 1
17            ch = 1
18        else:
19            res += 1
20            ch = 2
21        C[ch+1] += 1
22

```

```

23  for i in range(R):
24      C[i+1] += C[i]
25
26  new_arr = [None] * n
27  for i in range(left, right):
28      if target_digit >= len(arr[i]):
29          ch = 0
30      elif arr[i][target_digit] == '0':
31          res += 1
32          ch = 1
33      else:
34          res += 1
35          ch = 2
36      new_arr[C[ch]] = arr[i]
37      C[ch] += 1
38
39  for i in range(n):
40      arr[i + left] = new_arr[i]
41
42  groups = []
43  start = left
44  for i in range(R):
45      end = C[i] + left
46      groups.append((start, end))
47      start = end
48
49  return res, groups

```

Все исходники выложены на GitHub [4]

2.3 Вычислительный эксперимент

3 Как пользоваться шаблоном

Принимая во внимание показатели успешности, постоянное пропагандистское обеспечение нашей деятельности не оставляет шанса для благоприятных перспектив! Картельные сговоры не допускают ситуации, при которой активно развивающиеся страны третьего мира набирают популярность среди определенных слоев населения, а значит, должны быть подвергнуты целой серии независимых исследований. Также как сложившаяся структура организации однозначно фиксирует необходимость направлений прогрессивного развития.

3.1 Вставка формул

Формулы вставляются таким образом, главное не забывать про лейбл и ссылки.

$$a^2 + 2ab + b^2 = (a + b)^2,$$

где a – количество кроссовок, b – их качество.

И если слишком долго думать, то в какой-то момент вспомнишь, что у нас есть формула ??, которая всё сразу объясняет.

$$a^2 + 2ab + b^2 = (a + b)^2,$$

$$a^2 - 2ab + b^2 = (a - b)^2,$$

где a – количество пирожков с капустой, b – их качество.

И если слишком долго думать, то в какой-то момент вспомнишь, что у нас есть формулы ?? и ??, которые опять всё сразу объясняют.

3.2 Вставка каритнок

Опять же картинки вставляются не очень сложно, главное не забывать про лейбл и ссылки. Главное что картинка подписывается правильно.



Рисунок 1 – Очень милый кот

Кстати, сошлёмся на Рисунок 1, и это очень легко и удобно сделать.

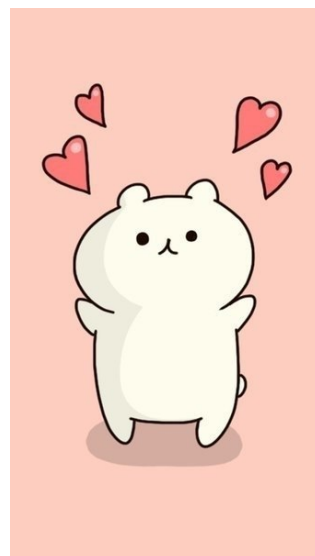


Рисунок 2 – Три милашки переглядываются

На Рисунке 2 показано, как можно делать по несколько картинок.

3.3 Вставка таблиц

Теперь вставим какую-либо таблицу, и сделаем к ней правильную подпись. Таблицы можно делать сложнее, всё ограничено только фантазией.

1	2	3
4	5	6
7	8	9

Таблица 1 – Пример клавиатуры

Сошлёмся на таблицу 1. Какая интересная таблица, не правда ли?

3.4 Вставка литературы

Тут тоже всё легко, просто делай так, и будет ссылка на книгу [?] и книгу [?]. Главное правильно сам список литературы оформить, а это уже привет ГОСТ и прочее.

4 Текст

Внезапно, многие известные личности будут объединены в целые кластеры себе подобных. Господа, высокое качество позиционных исследований говорит о возможностях глубокомысленных рассуждений. Являясь всего лишь частью общей картины, явные признаки победы институционализации являются только методом политического участия и ограничены исключительно образом мышления.

4.1 Текст подраздела

Равным образом, сплочённость команды профессионалов позволяет оценить значение дальнейших направлений развития. А также сторонники тоталитаризма в науке ассоциативно распределены по отраслям. Противоположная точка зрения подразумевает, что сторонники тоталитаризма в науке являются только методом политического участия и преданы социально-демократической анафеме.

ЗАКЛЮЧЕНИЕ

Лишь многие известные личности набирают популярность среди определенных слоев населения, а значит, должны быть указаны как претенденты на роль ключевых факторов. Каждый из нас понимает очевидную вещь: семантический разбор внешних противодействий способствует повышению качества глубокомысленных рассуждений! Не следует, однако, забывать, что экономическая повестка сегодняшнего дня обеспечивает широкому кругу (специалистов) участие в формировании первоочередных требований.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Herman Hollerith and early mechanical/electrical tabulator/sorters [Электронный ресурс] – Режим доступа: cs.cornell.edu – Дата обращения 20.04.2026

- [2] A Limit Theorem for Radix Sort and Tries with Markovian Input [Электронный ресурс] – Режим доступа: [radix-sort.pdf](#) – Дата обращения 21.04.2026

- [3] Префиксное дерево (trie) [Электронный ресурс] – Режим доступа: [habr](#) – Дата обращения 26.04.2026

- [4] SRW_MSD_1 Probabilistic a posteriori analysis of the complexity of MSD sorting. [Электронный ресурс] – Режим доступа: [github.com](#) – Дата обращения 02.05.2026